Refatoração com Padrões de Projeto GoF em Sistema Legado Solidity

Allo Protocol V2 - QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus Quixadá*

Aluno: Francisco Josias da Silva Batista - Matrícula 542167
Disciplina: Reuso de Software (2026.2)
Prof. Francisco Victor da Silva Pinheiro



Sumário da Apresentação

01 Sistema e Problemas do Legado

02 Os 4 Padrões GoF Aplicados

03 Padrão 1: Facade

04 Padrão 2: Strategy

05 Padrão 3: Factory Method

06 Padrão 4: State

07 Benefícios Arquiteturais

08 Resultados e Conclusão

Sistema e Problemas do Legado

Contexto Técnico

- **Allo Protocol V2**: alocação de recursos para DAOs na Ethereum
- **Stack**: Solidity 0.8.19, Foundry, Gitcoin Grants Stack
- **Arquitetura**: contratos modulares com padrão de estratégias

Problemas do Legado

- **God Class**: Allo.sol com 830+ linhas e múltiplas responsabilidades
- **Acoplamento alto**: negócio misturado com infraestrutura
- **Taxas hardcoded**: fee fixo em percentFee
- **Duplicação**: validações de estado repetidas em vários métodos
- **Interface dispersa**: operações comuns exigem várias transações

Os 4 Padrões GoF Aplicados

Padrão	Categoria	Problema Resolvido	Contrato Principal
Facade	Estrutural	Interface dispersa para criar pools	AlloStarterFacade.sol
Strategy	Comportamental	Taxas hardcoded no núcleo	FlatFeeStrategy.sol + IFeeStrategy
Factory Method	Criacional	Clonagem acoplada ao Allo.sol	StrategyFactory.sol + IStrategyFactory
State	Comportamental	Validações duplicadas de estado	LifecycleState.sol + estados concretos

Princípios aplicados: SOLID, baixo acoplamento, alta coesão e inversão de dependência

Padrão 1: Facade

Categoria: Estrutural

Problema

- 2 transações: createProfile + createPool
- Interação com múltiplos contratos
- Complexidade e custo de gás

Solução implementada

- AlloStarterFacade orquestra a criação
- Interface unificada de operações
- De 2 transações para 1

ANTES:

```
// Transação 1
bytes32 profileId = registry
  .createProfile(_nonce, _members,
                 _metadata.hash);

// Transação 2
uint256 poolId = allo.createPool(
  profileId, _strategy, _strategyData,
  _token, _amount);
```

DEPOIS:

```
// 1 transação via Facade
uint256 poolId =
  facade.initializeCampaign(
    _nonce, _members, _metadata,
    _strategy, _strategyData,
    _token, _amount);
```

Padrão 2: Strategy

Categoria: Comportamental

Problema

- percentFee fixo em 20% (hardcoded)
- Trocar o modelo exige editar o núcleo
- Viola o Princípio Aberto/Fechado

Solução implementada

- IFeeStrategy com calculateFee()
- FlatFeeStrategy: taxa fixa em %
- Injeção via setter onlyOwner

ANTES:

```
function _fundPool(...) internal {
  uint256 feeAmount = 0;
  if (percentFee > 0) {
    feeAmount =
      (_amount * percentFee) / 1e18;
  }
  uint256 amountAfterFee =
    _amount - feeAmount;
  _amounts[_poolId] += amountAfterFee;
}
```

DEPOIS:

```
interface IFeeStrategy {
  function calculateFee(uint256 amount)
    external view returns (
      uint256 fee, uint256 net);
}

// Allo.sol: _fundPool
(uint256 fee, uint256 net) =
  feeStrategy.calculateFee(_amount);
_amounts[_poolId] += net;
```

Padrão 3: Factory Method

Categoria: Criacional

Problema

- Clonagem EIP-1167 dentro de Allo.sol
- Bytecode maior no contrato principal
- Viola a Responsabilidade Única

Solução implementada

- Interface IStrategyFactory
- StrategyFactory.sol: só clonagem
- Contador _nonces na fábrica

ANTES:

```
function _createStrategyClone(
  address s, bytes memory args)
  internal returns (IStrategy) {
  bytes32 salt = keccak256(abi.encode(
    msg.sender, _nonces[msg.sender]++));
  address c = Clones
    .cloneDeterministic(s, salt);
  IStrategy(c).initialize(args);
  return IStrategy(c);
}
```

DEPOIS:

```
interface IStrategyFactory {
  function createStrategy(
    address impl, bytes memory args)
    external returns (IStrategy);
}

// Allo.sol: createPool(...)
IStrategy strategy = strategyFactory
  .createStrategy(_strategy,
                  _strategyData);
```

Padrão 4: State

Categoria: Comportamental

Problema

- Vários timestamps em DirectGrantsLite
- Checagens repetidas em cada método
- Cadeias if/else if temporais

ANTES:

```
function _allocate(...) internal {
  if (block.timestamp < regStart)
    revert REGISTRATION_NOT_STARTED();
  if (block.timestamp > regEnd)
    revert REGISTRATION_ENDED();
  if (block.timestamp > allocEnd)
    revert ALLOCATION_ENDED();
  // as mesmas checagens se
  // repetem em _distribute()
}
```

Solução implementada

- Enum PoolState (Open/Closed)
- Classe abstrata LifecycleState
- Um estado concreto por fase

DEPOIS:

```
enum PoolState {
  RegistrationOpen, RegistrationClosed }

abstract contract LifecycleState {
  function checkRegistration()
    external view virtual;
  // checkAllocation, checkDistribution
}
// _allocate() passa a delegar:
_currentState().checkRegistration();
```

Benefícios Arquiteturais

1. Reutilização

- Factory reutilizável em outros protocolos
- Estratégias de taxas plugáveis
- Estados independentes do negócio

2. Extensibilidade

- TieredFee/DynamicFee sem tocar no núcleo
- Novos estados sem refatorar métodos
- Novas fábricas: proxy, beacon

3. Manutenibilidade

- ~30% menos duplicação de código
- Menor complexidade ciclomática
- Responsabilidades separadas (SRP)

4. Conformidade com SOLID

- **SRP**: uma responsabilidade por contrato
- **OCP**: aberto para extensão, fechado para modificação
- **LSP**: estados e estratégias intercambiáveis
- **ISP**: interfaces granulares e específicas
- **DIP**: dependência de interfaces, não de implementações

5. Qualidade e Testes

- 1147 testes Foundry passando (100%)
- 100% de cobertura nos novos módulos
- Zero regressões após a refatoração

Resultados e Conclusão

Métrica	Antes	Depois	Melhoria
Linhas em Allo.sol	830+	~650	-22%
Transações para criar pool	2	1	-50%
Modelos de taxas	1 (hardcoded)	N (plugáveis)	+∞ flexibilidade
Validações duplicadas	~40 linhas	0	-100%
Testes automatizados	1133	1147	+14 novos testes

Validação técnica

- Suíte completa: 1147 testes (Foundry)
- 32 testes unitários novos para os padrões
- 100% de cobertura nos novos contratos
- Sem quebra de compatibilidade

Conclusão

A refatoração demonstrou a aplicação prática dos padrões GoF em Solidity: arquitetura mais modular e coesa, flexibilidade para evoluir o protocolo, menor risco de manutenção e aderência aos princípios SOLID.

Muito Obrigado!

Universidade Federal do Ceará - *Campus Quixadá*

Prof. Francisco Victor da Silva Pinheiro
Aluno: Francisco Josias da Silva Batista - Matrícula 542167

